Chapter 4. Common kubectl Commands

The `kubectl` command-line utility is a powerful tool, and in the following chapters, you will use it to create objects and interact with the Kubernetes API. Before that, however, it makes sense to go over the basic `kubectl` commands that apply to all Kubernetes objects.

Namespaces

Kubernetes uses *namespaces* to organize objects in the cluster. You can think of each namespace as a folder that holds a set of objects. By default, the `kubectl` command-line tool interacts with the `default` namespace. If you want to use a different namespace, you can pass `kubectl` the `--namespace` flag. For example, `kubectl --namespace=mystuff` references objects in the `mystuff` namespace. You can also use the shorthand `-n` flag if you're feeling concise. If you want to interact with all namespaces—for example, to list all Pods in your cluster—you can pass the `--all-namespaces` flag.

Contexts

If you want to change the default namespace more permanently, you can use a *context*. This gets recorded in a `kubectl` configuration file, usually located at *$HOME/.kube/config*. This configuration file also stores how to both find and authenticate to your cluster. For example, you can create a context with a different default namespace for your `kubectl` commands using:

```
$ kubectl config set-context my-context --namespace=mystuff
```

This creates a new context, but it doesn't actually start using it yet. To use this newly created context, you can run:

```
$ kubectl config use-context my-context
```

Contexts can also be used to manage different clusters or different users for authenticating to those clusters using the `--users` or `--clusters` flags with the `set-context` command.

Viewing Kubernetes API Objects

Everything contained in Kubernetes is represented by a RESTful resource. Throughout this book, we refer to these resources as *Kubernetes objects*. Each Kubernetes object exists at a unique HTTP path; for example, *https://your-k8s.com/api/v1/namespaces/default/pods/my-pod* leads to the representation of a Pod in the default namespace named `my-pod`. The `kubectl` command makes HTTP requests to these URLs to access the Kubernetes objects that reside at these paths.

The most basic command for viewing Kubernetes objects via `kubectl` is `get`. If you run `kubectl get <resource-name>`, you will get a listing of all resources in the current namespace. If you want to get a specific resource, you can use `kubectl get <resource-name> <obj-name>`.

By default, `kubectl` uses a human-readable printer for viewing the responses from the API server, but this human-readable printer removes many of the details of the objects to fit each object on one terminal line. One way to get slightly more information is to add the `-o wide` flag, which gives more details, on a longer line. If you want to view the complete object, you can also view the objects as raw JSON or YAML using the `-o json` or `-o yaml` flags, respectively.

A common option for manipulating the output of `kubectl` is to remove the headers, which is often useful when combining `kubectl` with Unix pipes (e.g., `kubectl ... | awk ...`). If you specify the `--no-headers` flag, `kubectl` will skip the headers at the top of the human-readable table.

Another common task is extracting specific fields from the object. `kubectl` uses the JSONPath query language to select fields in the returned object. The complete details of JSONPath are beyond the scope of this chapter, but as an example, this command will extract and print the IP address of the specified Pod:

```
$ kubectl get pods my-pod -o jsonpath --template={.status.podIP}
```

You can also view multiple objects of different types by using a comma separated list of types, for example:

```
$ kubectl get pods,services
```

This will display all Pods and services for a given namespace.

If you are interested in more detailed information about a particular object, use the `describe` command:

```
$ kubectl describe <resource-name> <obj-name>
```

This will provide a rich multiline human-readable description of the object as well as any other relevant, related objects and events in the Kubernetes cluster.

If you would like to see a list of supported fields for each supported type of Kubernetes object, you can use the `explain` command:

```
$ kubectl explain pods
```

Sometimes you want to continually observe the state of a particular Kubernetes resource to see changes to the resource when they occur. For example, you might be waiting for your application to restart. The `--watch` flag enables this. You can add this flag to any `kubectl get` command to continuously monitor the state of a particular resource.

Creating, Updating, and Destroying Kubernetes Objects

Objects in the Kubernetes API are represented as JSON or YAML files. These files are either returned by the server in response to a query or posted to the server as part of an API request. You can use these YAML or JSON files to create, update, or delete objects on the Kubernetes server.

Let's assume that you have a simple object stored in *obj.yaml*. You can use `kubectl` to create this object in Kubernetes by running:

```
$ kubectl apply -f obj.yaml
```

Notice that you don't need to specify the resource type of the object; it's obtained from the object file itself.

Similarly, after you make changes to the object, you can use the `apply` command again to update the object:

```
$ kubectl apply -f obj.yaml
```

The `apply` tool will only modify objects that are different from the current objects in the cluster. If the objects you are creating already exist in the cluster, it will simply exit successfully without making any changes. This makes it useful for loops where you want to ensure the state of the cluster matches the state of the filesystem. You can repeatedly use `apply` to reconcile state.

If you want to see what the `apply` command will do without actually making the changes, you can use the `--dry-run` flag to print the objects to the terminal without actually sending them to the server.

NOTE

If you feel like making interactive edits instead of editing a local file, you can instead use the `edit` command, which will download the latest object state and then launch an editor that contains the definition:

```
$ kubectl edit <resource-name> <obj-name>
```

After you save the file, it will be automatically uploaded back to the Kubernetes cluster.

The `apply` command also records the history of previous configurations in an annotation within the object. You can manipulate these records with the `edit-last-applied`, `set-last-applied`, and `view-last-applied` commands. For example:

```
$ kubectl apply -f myobj.yaml view-last-applied
```

will show you the last state that was applied to the object.

When you want to delete an object, you can simply run:

```
$ kubectl delete -f obj.yaml
```

It is important to note that `kubectl` will not prompt you to confirm the deletion. Once you issue the command, the object *will* be deleted.

Likewise, you can delete an object using the resource type and name:

```
$ kubectl delete <resource-name> <obj-name>
```

Labeling and Annotating Objects

Labels and annotations are tags for your objects. We'll discuss the differences in Chapter 6, but for now, you can update the labels and annotations on any Kubernetes object using the `label` and `annotate` commands. For example, to add the `color=red` label to a Pod named `bar`, you can run:

```
$ kubectl label pods bar color=red
```

The syntax for annotations is identical.

By default, `label` and `annotate` will not let you overwrite an existing label. To do this, you need to add the `--overwrite` flag.

If you want to remove a label, you can use the `<label-name>-` syntax:

```
$ kubectl label pods bar color-
```

This will remove the `color` label from the Pod named `bar`.

Debugging Commands

`kubectl` also makes a number of commands available for debugging your containers. You can use the following to see the logs for a running container:

```
$ kubectl logs <pod-name>
```

If you have multiple containers in your Pod, you can choose the container to view using the `-c` flag.

By default, `kubectl logs` lists the current logs and exits. If you instead want to continuously stream the logs back to the terminal without exiting, you can add the `-f` (follow) command-line flag.

You can also use the `exec` command to execute a command in a running container:

```
$ kubectl exec -it <pod-name> -- bash
```

This will provide you with an interactive shell inside the running container so that you can perform more debugging.

If you don't have bash or some other terminal available within your container, you can always `attach` to the running process:

```
$ kubectl attach -it <pod-name>
```

The `attach` command is similar to `kubectl logs` but will allow you to send input to the running process, assuming that process is set up to read from standard input.

You can also copy files to and from a container using the `cp` command:

```
$ kubectl cp <pod-name>:</path/to/remote/file> </path/to/local/file>
```

This will copy a file from a running container to your local machine. You can also specify directories, or reverse the syntax to copy a file from your local machine back out to the container.

If you want to access your Pod via the network, you can use the `port-forward` command to forward network traffic from the local machine to the Pod. This enables you to securely tunnel network traffic through to containers that might not be exposed anywhere on the public network. For example, the following command:

```
$ kubectl port-forward <pod-name> 8080:80
```

opens up a connection that forwards traffic from the local machine on port 8080 to the remote container on port 80.

NOTE

You can also use the `port-forward` command with services by specifying `services/<service-name>` instead of `<pod-name>`, but note that if you do port-forward to a service, the requests will only ever be forwarded to a single Pod in that service. They will not go through the service load balancer.

If you want to view Kubernetes events, you can use the `kubectl get events` command to see a list of the latest 10 events on all objects in a given namespace:

```
$ kubectl get events
```

You can also stream events as they happen by adding `--watch` to the `kubectl get events` command. You may also wish to include `-A` to see events in all namespaces.

Finally, if you are interested in how your cluster is using resources, you can use the `top` command to see the list of resources in use by either nodes or Pods. This command:

```
$ kubectl top nodes
```

will display the total CPU and memory in use by the nodes in terms of both absolute units (e.g., cores) and percentage of available resources (e.g., total number of cores). Similarly, this command:

```
$ kubectl top pods
```

will show all Pods and their resource usage. By default it only displays Pods in the current namespace, but you can add the `--all-namespaces` flag to see resource usage by all Pods in the cluster.

These `top` commands only work if a metrics server is running in your cluster. Metrics servers are present in nearly every managed Kubernetes environment and many unmanaged environments as well. But if these commands fail, it may be because you need to install a metrics server.

Cluster Management

The `kubectl` tool can also be used to manage the cluster itself. The most common action that people take to manage their cluster is to cordon and drain a particular node. When you *cordon* a node, you prevent future Pods from being scheduled onto that machine. When you *drain* a node, you remove any Pods that are currently running on that machine. A good example use case for these commands would be removing a physical machine for repairs or upgrades. In that scenario, you can use `kubectl cordon` followed by `kubectl drain` to safely remove the machine from the cluster. Once the machine is repaired, you can use `kubectl uncordon` to re-enable Pods scheduling onto the node. There is no `undrain` command; Pods will naturally get scheduled onto the empty node as they are created. For something quick affecting a node (e.g., a machine reboot), it is generally unnecessary to cordon or drain; it's only necessary if the machine will be out of service long enough that you want the Pods to move to a different machine.

Command Autocompletion

`kubectl` supports integration with your shell to enable tab completion for both commands and resources. Depending on your environment, you may need to install the `bash-completion` package before you activate command autocompletion. You can do this using the appropriate package manager:

```
# macOS
$ brew install bash-completion

# CentOS/Red Hat
$ yum install bash-completion

# Debian/Ubuntu
$ apt-get install bash-completion
```

When installing on macOS, make sure to follow the instructions from `brew` about how to activate tab completion using your *${HOME}/.bash_profile*.

Once `bash-completion` is installed, you can temporarily activate it for your terminal using:

```
$ source <(kubectl completion bash)
```

To make this automatic for every terminal, add it to your *${HOME}/.bashrc* file:

```
$ echo "source <(kubectl completion bash)" >> ${HOME}/.bashrc
```

If you use `zsh`, you can find similar instructions online.

Alternative Ways of Viewing Your Cluster

In addition to `kubectl`, there are other tools for interacting with your Kubernetes cluster. For example, there are plug-ins for several editors that integrate Kubernetes and the editor environment, including:

- Visual Studio Code
- IntelliJ
- Eclipse

If you are using a managed Kubernetes service, most of them also feature a graphical interface to Kubernetes integrated into their web-based user experience. Managed Kubernetes in the public cloud also integrates with sophisticated monitoring tools that can help you gain insights into how your applications are running.

There are also several open source graphical interfaces for Kubernetes including Rancher Dashboard and the Headlamp project.

Summary

`kubectl` is a powerful tool for managing your applications in your Kubernetes cluster. This chapter has illustrated many of the common uses for the tool, but `kubectl` has a great deal of built-in help available. You can start viewing this help with:

```
$ kubectl help
```

or:

```
$ kubectl help <command-name>
```

Previous chapter

< 3. Deploying a Kubernetes Cluster

Next chapter

5. Pods >